

minishell

Fiches de soutenance

42 Lausanne · aalvard & dbomfim- · septembre 2026

À quoi sert ce document

Il ne remplace pas votre code, il vous rend capables de le défendre. Un correcteur de 42 ne lit presque jamais tout le dépôt : il tape des commandes, il regarde ce qui sort, et il pose des questions. Ce document suit exactement cette logique.

Partie 1 : la grille d'évaluation officielle, section par section. Pour chaque section, ce que le correcteur tape, ce que votre shell répond, où c'est géré dans votre code, et le piège.

Partie 2 : le code module par module, avec les extraits qui comptent et le *pourquoi* de chaque choix. C'est la partie à lire en croisé. Axel révise l'exécuteur de Davi, Davi révise le lexer d'Axel.

Partie 3 : la soutenance. Qui a écrit quoi, les questions pièges avec leurs réponses, le résultat de la campagne adversariale du 23 septembre, et la checklist du jour J.

Les trois choses qui mettent zéro, avant tout le reste

1. Un segfault. La grille dit mot pour mot : *for the duration of the defense, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0*. Un seul crash, n'importe quand, et c'est fini.

2. Une fuite mémoire. Le correcteur a le droit de sortir valgrind. Vos fuites à vous doivent être à zéro.

3. Une erreur de norme, ou plus d'une variable globale. Flag, et fin de l'évaluation.

Tout le reste se discute. Ces trois-là, non.

Partie 1

La grille d'évaluation, section par section

Source : la fiche officielle de 42evalhub pour le projet minishell, dans sa version de septembre 2026. Les 21 sections ci-dessous sont celles que le correcteur doit cocher. Lisez-les comme un script, c'est littéralement ce qu'il va faire.

1. Compile

```
# make -n ne compile pas, il montre ce qui SERAIT fait.
# Il faut donc partir d'un projet nettoye, sinon il ne montre rien.
make fclean
make -n | grep -- -Wall

# la compilation réelle, c'est celle-ci
make
make # une seconde fois : Nothing to be done
```

Ce que votre shell répond

Les trois drapeaux **-Wall -Wextra -Werror** sont sur la ligne **CFLAGS** du Makefile. Un **make** relancé juste après un premier **make** affiche *Nothing to be done for 'all'* : pas de relink.

Où c'est géré, auteur : Axel

Makefile. La règle **%o: %c** compare les dates, donc aucun objet n'est reconstruit inutilement. Le bloc **ifeq (\$(shell uname), Darwin)** ajoute les chemins de la readline de Homebrew, parce que macOS ne fournit que libedit, dont l'en-tête ne déclare pas **rl_replace_line**.

Le piège

Deux choses à savoir avant que le correcteur tape. **Premièrement**, **make -n** ne compile rien, il affiche seulement ce qui serait exécuté. Sur un projet déjà construit il répond *Nothing to be done* et le **grep** ne trouve aucun drapeau, ce qui peut faire croire qu'ils manquent. Il faut un **make fclean** avant. **Deuxièmement**, **make -n** ne marche pas non plus ici : le **rm** étant simulé, **make** considère les objets encore à jour et n'affiche aucune ligne de compilation. Si le correcteur bloque là-dessus, montrez-lui directement la ligne 4 du Makefile.

Ce que vous répondez

La ligne 4 du Makefile dit **CFLAGS = -Wall -Wextra -Werror**, et rien nulle part ne les retire. Un **make fclean** puis **make -n** le confirme ligne par ligne.

2. Simple Command et variables globales

```
minishell$ /bin/ls
minishell$ ls
minishell$ # ligne vide, Entree
minishell$ \t # espaces et tabulations seules
```

Ce que votre shell répond

Les commandes s'exécutent. Une ligne vide ou blanche ne produit rien et laisse **\$?** inchangé. Il y a **exactement une variable globale**, **int g_signal**, dans **src/signals/signals.c**.

Où c'est géré, auteur : Davi (main, signaux), Axel (run_line)

src/main.c pour la boucle, **src/utills/ut_init_data.c** pour **run_line**, **src/signals/signals.c** pour la globale. Sur une ligne vide, le lexer renvoie une liste NULL et **run_line** sort immédiatement.

Le piège

La question *combien de globales et pourquoi* est écrite dans la grille. Il faut une réponse précise, pas *une seule*, comme demandé.

Ce que vous répondez

Une seule : un int qui reçoit le numéro du signal. Un gestionnaire de signal ne peut ni recevoir de paramètre ni rendre de valeur, il ne peut écrire que dans une variable de portée globale. On y met uniquement le numéro du signal, jamais un pointeur ni une donnée du shell, ce que le sujet interdit explicitement. Le code de sortie 130 est ensuite calculé à partir de cette variable dans main.

3. Arguments

```
minishell$ /bin/ls -la
minishell$ /usr/bin/wc -l Makefile
minishell$ grep include includes/minishell.h
```

Ce que votre shell répond

Les arguments sont découpés sur les espaces **et les tabulations**, puis transmis tels quels à **execve** via **cmd->argv**.

Où c'est géré, auteur : Axel (lexer, parser), Davi (execve)

src/lexer/lex_tokenize.c découpe, **src/parser/parse_word.c** construit le tableau argv, **src/executor/exec_child.c** appelle execve.

Ce que vous répondez

Le lexer traite l'espace et la tabulation comme des séparateurs. Davi a ajouté la tabulation le 22 septembre, c'était un oubli trouvé en testant.

4. echo

```
minishell$ echo
minishell$ echo bonjour
minishell$ echo -n sans retour
minishell$ echo -nnnn aussi
minishell$ echo -n -n encore
minishell$ echo -N pas une option
minishell$ echo -xn pas une option non plus
```

Ce que votre shell répond

-n et ses répétitions sont consommées tant que l'argument ne contient que des **n** après le tiret. **-N**, **-xn**, **-n-** et le tiret seul sont affichés comme du texte, comme dans bash.

Où c'est géré, auteur : Axel

src/builtins/bi_echo.c. La boucle avance tant que l'argument commence par - et n'est suivi que de **n**.

Ce que vous répondez

60 assertions dans test_builtins comparent notre echo à celui de bash, argument par argument.

5. exit

```
minishell$ exit 42 # relancer ensuite
minishell$ exit abc
minishell$ exit 1 2
minishell$ exit 256
minishell$ exit -1
```

Ce que votre shell répond

exit 42 sort avec 42. **exit abc** affiche *numeric argument required* et sort avec 2. **exit 1 2** affiche *too many arguments*, **ne sort pas** et rend 1. **exit 256** sort avec 0 et **exit -1** avec 255 : le code est pris modulo 256.

Où c'est géré, auteur : Axel

src/builtins/bi_exit.c. Le drapeau **shell->want_exit** fait sortir la boucle de **main**, ce qui permet de tout libérer avant de rendre la main.

Le piège

Le bash 3.2 d'Apple ne traite pas **exit abc def** comme bash 5. Il regarde d'abord le nombre d'arguments, bash 5 regarde d'abord la validité du premier. Notre shell suit bash 5, qui est ce qui tourne sur les machines de l'école.

Ce que vous répondez

On ne fait jamais exit() depuis un builtin, sauf exit lui-même. On pose un drapeau et on sort de la boucle, sinon on saute la libération finale et on fuit tout.

6. Return value of a process

```
minishell$ /bin/ls ; echo $?
minishell$ /bin/ls filequinexistepas ; echo $?
minishell$ commandebidon ; echo $?
minishell$ expr $? + $?
minishell$ /etc ; echo $?
```

Ce que votre shell répond

0 en cas de succès, 2 pour ls sur un fichier absent, **127** pour une commande introuvable, **126** pour un répertoire ou un fichier sans droit d'exécution, **128 + N** quand le processus meurt d'un signal. **expr \$? + \$?** fonctionne.

Où c'est géré, auteur : Davi (attente, codes), Axel (\$? dans l'expander)

src/executor/exec_wait.c : **WEXITSTATUS** si le fils est sorti normalement, **128 + WTERMSIG** s'il a été tué. **src/executor/exec_single.c** pour 126 et 127 avant le fork. **\$?** est lu depuis **shell->last_exit** par l'expander.

Ce que vous répondez

*last_exit vit dans la structure **t_shell**, pas dans une globale. C'est justement pour ça qu'une seule globale suffit.*

7. Signals

```
minishell$ ctrl-C # ligne vide
minishell$ ctrl-backslash # ne doit rien faire
minishell$ ctrl-D # quitte, RELANCER
minishell$ du texte ctrl-C # puis Entree : rien ne doit s'executer
minishell$ cat ctrl-C # puis echo $? -> 130
minishell$ cat ctrl-backslash # Quit (core dumped), $? -> 131
minishell$ cat ctrl-D # cat se termine
```

Ce que votre shell répond

Tous ces cas sont conformes, vérifiés le 23 septembre dans un vrai terminal sur Mac et pilotés par pseudo-terminal sous Linux. **ctrl-C** pendant un heredoc annule le heredoc et rend le prompt.

Où c'est géré, auteur : Davi

`src/signals/signals.c` contient trois jeux de gestionnaires. Au prompt, SIGINT écrit un retour à la ligne puis appelle `rl_on_new_line`, `rl_replace_line("", 0)` et `rl_redisplay` : c'est ce trio qui vide le tampon de readline. Pendant l'exécution, SIGINT et SIGQUIT sont ignorés par le père pour que seul le fils meure.

`src/signals/signals_heredoc.c` pose un gestionnaire sans `SA_RESTART`, ce qui fait remonter `EINTR` du `read` bloquant et annule le heredoc.

Le piège

Le point le plus testé de toute la grille : *the buffer should be clean too, press Enter to make sure nothing from the previous line is executed*. Sans `rl_replace_line`, la ligne tapée reste dans readline et part à l'exécution au prochain Entrée.

Ce que vous répondez

`rl_catch_signals = 0` dans `main` : on désactive la gestion interne de readline pour poser la nôtre. Le 130, c'est 128 + SIGINT, calculé dans `check_prompt_signal` à partir de la globale.

8. Double Quotes

```
minishell$ echo "cat lol.c | cat > lol.c"
minishell$ echo "un deux"
minishell$ echo "a"b"c"
```

Ce que votre shell répond

Le contenu entre guillemets est traité comme un seul mot. Les pipes et les chevrons à l'intérieur ne sont jamais interprétés. Les espaces multiples sont conservés.

Où c'est géré, auteur : Axel

`src/lexer/lex_quotes.c` implémente un automate à trois états : NORMAL, SQUOTE, DQUOTE. Tant qu'on est dans un état coté, `|` et `>` sont accumulés comme des caractères ordinaires et ne sont pas reconnus comme opérateurs.

Ce que vous répondez

Les quotes sont conservées dans le token jusqu'à l'expander. Si on les enlevait dans le lexer, on ne saurait plus, au moment d'expander, si un dollar était entre simples quotes ou non.

9. Single Quotes

```
minishell$ echo '$USER'
minishell$ echo ''
minishell$ echo 'a | b > c'
minishell$ echo ' espaces '
```

Ce que votre shell répond

`echo '$USER'` affiche `$USER` littéralement. Rien n'est interprété entre simples quotes, ni le dollar, ni les pipes, ni les chevrons.

Où c'est géré, auteur : Axel

`src/expander/exp_var_replace.c`. La fonction `update_state` rejoue le même automate que le lexer, et l'expansion n'a lieu que si `state != STATE_SQUOTE`.

Ce que vous répondez

Le même automate tourne deux fois, dans le lexer puis dans l'expander. C'est volontaire : le lexer découpe sans comprendre, l'expander comprend sans redécouper.

10 à 12. env, export, unset

```
minishell$ env
minishell$ export TEST=42
minishell$ env | grep TEST
minishell$ export TEST=43
minishell$ unset TEST
minishell$ env | grep TEST # plus rien
minishell$ export 1BAD=x # nom invalide
```

Ce que votre shell répond

L'environnement est copié au démarrage dans une liste chaînée. **export** crée ou remplace, **unset** retire, **env** affiche. Un identifiant invalide est refusé avec un message.

Où c'est géré, auteur : Davi

src/env/env_init.c copie **envp** en liste chaînée de **t_env** avec une clé, une valeur et un drapeau exporté. **src/env/env_to_array.c** refabrique le tableau au moment du **execve**. Les builtins sont dans **src/builtins/bi_export.c**, **bi_unset.c** et **bi_env.c**.

Le piège

La liste chaînée est une copie, elle ne pointe jamais sur **envp** : ce tableau appartient au système et ne doit pas être modifié.

Ce que vous répondez

Le drapeau exported distingue une variable créée par export d'une variable interne. env n'affiche que celles qui sont exportées, comme bash.

13 à 15. cd, pwd, chemins relatifs

```
minishell$ cd /tmp ; pwd
minishell$ cd .. ; pwd
minishell$ cd . ; pwd
minishell$ cd - ; pwd
minishell$ cd ../../.. ; /bin/ls
minishell$ cd /nexistepas ; echo $?
```

Ce que votre shell répond

cd sans argument va dans HOME, **cd -** revient au précédent et affiche le chemin, **PWD** et **OLDPWD** sont mis à jour. Une cible invalide affiche l'erreur système et rend 1.

Où c'est géré, auteur : Davi (cd), Axel (pwd)

src/builtins/bi_cd.c : **get_target** choisit la cible, **chdir** se déplace, puis **update_pwd** récupère le chemin réel avec **getcwd** et met à jour les deux variables. **src/builtins/bi_pwd.c** utilise **getcwd(NULL, 0)** et libère le résultat.

Le piège

getcwd rend le chemin **physique**. Après un **cd** à travers un lien symbolique, **pwd** affiche la cible réelle là où bash garde le chemin logique. C'est un écart assumé, le sujet n'exige pas le chemin logique.

Ce que vous répondez

On appelle getcwd après chaque chdir plutôt que de bricoler la chaîne, parce que c'est le noyau qui a la vérité sur l'endroit où on se trouve.

16. Environment path

```
minishell$ ls
```

```
minishell$ unset PATH ; ls ; echo $? # 127
minishell$ export PATH=/dir1:/dir2 ; macommande
```

Ce que votre shell répond

Une commande sans slash est cherchée dans les répertoires du PATH, de gauche à droite. Sans PATH, on rend 127 avec *command not found*.

Où c'est géré, auteur : Davi (écriture initiale), Axel (correction du 23 septembre)

src/executor/exec_path.c. resolve_path distingue deux cas : un nom contenant un slash est testé directement, sinon **resolve_in_path** parcourt les répertoires.

Le piège

C'est ici qu'un bug a été trouvé le 23 septembre. La recherche s'arrêtait au premier fichier du bon nom, même non exécutable. Un fichier non exécutable ou un répertoire homonyme situé dans un répertoire à gauche masquait donc le vrai binaire. Corrigé : on continue jusqu'à trouver un exécutable.

Ce que vous répondez

Si aucun candidat n'est exécutable, on rend Permission denied (126) si on en a croisé un, sinon command not found (127). C'est exactement la sémantique de bash, et c'est verrouillé par test_robustness.

17. Redirection

```
minishell$ echo salut > f ; cat f
minishell$ echo encore >> f ; cat f
minishell$ wc -l < f
minishell$ cat << EOF
minishell$ echo a > f1 > f2 > f3
minishell$ cat < fichierabsent ; echo $?
```

Ce que votre shell répond

Les quatre formes fonctionnent. Avec plusieurs redirections de sortie, toutes les cibles sont ouvertes et seule la dernière reçoit les données, comme bash. Un fichier d'entrée absent affiche l'erreur système et rend 1.

Où c'est géré, auteur : Davi

src/redirections/redir_files.c : open_redir choisit les bons drapeaux, **O_TRUNC** pour le chevron simple et **O_APPEND** pour le double, puis **apply_redirs** fait le **dup2** et referme tout de suite le descripteur d'origine.

Le piège

Le **close(fd)** juste après le **dup2** est obligatoire. Sans lui, chaque redirection laisse un descripteur ouvert et on finit par épuiser la table de descripteurs du processus.

Ce que vous répondez

Pour un builtin seul, on sauvegarde stdin et stdout avec dup avant d'appliquer les redirections, et on les restaure après. Sinon un echo vers un fichier casserait la sortie du shell pour toute la session.

18. Pipes

```
minishell$ cat Makefile | grep SRC | wc -l
minishell$ ls fichierabsent | grep bla | more
minishell$ cat | cat | ls
minishell$ ls | wc -l > out ; cat out
minishell$ cd /tmp | echo hi ; pwd # le cwd ne change pas
```

Ce que votre shell répond

Les pipelines de longueur quelconque fonctionnent. **\$?** vaut le code de la **dernière** commande. Un builtin dans un pipe s'exécute dans un fils, donc **cd** et **export** n'ont aucun effet sur le shell père, exactement comme bash.

Où c'est géré, auteur : Davi

src/executor/exec_pipeline.c : pour chaque étage, le tube est créé **avant** le fork, le fils branche ses descripteurs avec **child_dup_pipes**, le père referme son extrémité immédiatement. Tous les forks sont faits **avant** la première attente.

Le piège

Forker tous les étages avant d'attendre est obligatoire. Si on attendait le premier fils avant de forker le second, un pipeline dont la première commande produit beaucoup de données remplirait le tube et se bloquerait, puisque personne ne lit à l'autre bout.

Ce que vous répondez

wait_all garde le code du dernier fils attendu. C'est la sémantique de bash sans pipefail.

19. Go Crazy and history

```
minishell$ du texte ctrl-C puis Entree
minishell$ fleche haut, fleche bas
minishell$ dsbksdgbksdghsd
minishell$ cat | cat | ls
minishell$ une commande avec 10 000 arguments
```

Ce que votre shell répond

L'historique fonctionne via **add_history**. Une commande inconnue affiche une erreur et rend 127 sans planter. **cat | cat | ls** se comporte normalement. Les lignes très longues et les milliers d'arguments sont supportés depuis la correction du 23 septembre.

Où c'est géré, auteur : Davi (main), Axel (tampon dynamique)

src/main.c pour **add_history**, **src/lexer/lex_tokenize_utils.c** pour le tampon dynamique.

Le piège

C'est ici que la campagne adversariale a trouvé le segfault. Le tampon du lexer était figé à 4096 octets sur la pile, et tout mot plus long écrivait à côté. La section demande explicitement *try to execute a long command with a ton of arguments*. Voir la partie 3.

Ce que vous répondez

Une ligne n'est mise dans l'historique que si elle n'est pas vide, sinon on le pollue avec des lignes blanches.

20 et 21. Environment variables et la section Surprise

```
minishell$ echo $USER
minishell$ echo "$USER"
minishell$ echo '$USER'
minishell$ echo "'$USER'"
minishell$ echo '"$USER"'
minishell$ echo $INEXISTANT
minishell$ echo a"b"c'd'$USER
```

Ce que votre shell répond

Ligne par ligne : la valeur, la valeur, le texte littéral, la valeur entourée de simples quotes, le texte **"\$USER"** littéral, une ligne vide, puis la concaténation **abcd** suivie de la valeur.

Où c'est géré, auteur : Axel

`src/expander/exp_run.c` orchestre. D'abord l'**expansion** sur tous les mots, ensuite seulement le **retrait des quotes** via `exp_strip_tokens`. L'ordre est obligatoire.

Le piège

La section Surprise existe uniquement pour vérifier cet ordre. Si vous retirez les quotes avant d'expanser, **echo '\$USER'** affiche la valeur au lieu du texte, et vous perdez la section.

Ce que vous répondez

Expansion d'abord, retrait des quotes ensuite. C'est la seule chose à retenir de tout l'expander, et c'est pour ça que `exp_run` tient en trois lignes.

Récapitulatif de la partie 1, à relire la veille

Les quatre sections où l'on perd des points sans s'en rendre compte

Signaux. Le tampon de readline doit être vide après ctrl-C. Tapez du texte, ctrl-C, Entrée : rien ne doit s'exécuter.

Environment path. Le correcteur construit un PATH à deux répertoires et vérifie l'ordre. Un fichier non exécutable ou un répertoire homonyme ne doit pas masquer le vrai binaire.

Go crazy. Une commande avec beaucoup d'arguments, une ligne très longue. C'est le test qui cherche le segfault.

Surprise. `echo "$USER"` affiche la valeur entre simples quotes, `echo "$USER"` affiche le texte. Ces deux lignes vérifient à elles seules que l'expansion précède le retrait des quotes.

Les trois chiffres à connaître par cœur

127 commande introuvable, **126** répertoire ou droit d'exécution manquant, **128 plus N** mort par signal, donc 130 pour ctrl-C et 131 pour ctrl-backslash.

La phrase à ne jamais prononcer

« Je ne sais pas, c'est l'autre qui l'a fait. » La partie 2 existe pour que cette phrase ne soit jamais nécessaire.

Partie 2

Le code, module par module

Cette partie se lit **en croisé**. Axel connaît le lexer, le parser et l'expander : il doit réviser l'exécuteur, les heredocs et les signaux. Davi connaît l'exécuteur : il doit réviser le lexer et l'expander. Le correcteur choisit qui il interroge, et il choisit souvent celui qui n'a pas écrit le fichier qu'il pointe.

Le chemin complet d'une ligne

```
l'utilisateur tape une ligne
|
v
+-----+
| readline() | <-- historique, fleches, ctrl-C
+-----+
| "echo \"$USER\" | wc -c > out"
v
+-----+
| syntax_check_line | quote non fermee ? -> erreur, $? = 2
+-----+
|
v
+-----+
| LEXER | automate a 3 etats. Les quotes sont
| lex_tokenize / lex_quotes | CONSERVEES dans le token.
+-----+
| [WORD echo] [WORD "$USER"] [PIPE] [WORD wc] ...
v
+-----+
| syntax_check_tokens | | en bord, > sans cible -> $? = 2
+-----+
|
v
+-----+
| EXPANDER | 1. expansion de $VAR et $?
| exp_run = expand + strip | 2. PUIS retrait des quotes
+-----+ l'ordre est obligatoire
| [WORD echo] [WORD alvrd] [PIPE] [WORD wc] ...
v
+-----+
| PARSEUR | liste chaine de t_cmd,
| parse_input / parse_word | redirections accrochees
+-----+
| cmd1(echo alvrd) -> cmd2(wc -c, > out)
v
+-----+
| collect_heredocs | lit les << AVANT d'executer
+-----+
|
v
+-----+
| EXECUTOR | fork, pipe, dup2, execve, wait
| un builtin seul reste | sinon cd et export seraient
| dans le pere | perdus a chaque commande
+-----+
|
v
shell->last_exit mis a jour
|
v
free des tokens, de l'AST et de la ligne
|
v
retour a readline()
```

Le lexer, par Axel

Son travail : transformer une chaîne en liste de tokens. Il ne comprend rien à ce qu'il découpe. Il sait seulement reconnaître les séparateurs, les opérateurs, et se souvenir s'il est à l'intérieur d'une paire de quotes.

L'automate à trois états

```
28 void lex_handle_quote_char(t_lex *lex, char c)
29 {
30     if (!lex_in_quote(lex))
31     {
32         if (c == '"')
33             lex->state = STATE_DQUOTE;
34         else
35             lex->state = STATE_SQUOTE;
36         lex->token_active = 1;
37         lex->has_quotes = 1;
38         append_char(lex, c);
39         return ;
40     }
41     if ((lex->state == STATE_SQUOTE && c == '\\')
42         || (lex->state == STATE_DQUOTE && c == '"'))
43     {
44         lex->state = STATE_NORMAL;
45         append_char(lex, c);
46         return ;
47     }
48     lex->token_active = 1;
49     append_char(lex, c);
50 }
```

Ce que ça fait

Trois situations. Hors quote, un guillemet ouvre l'état correspondant. Dans une quote, **seul** le même caractère la referme. Tout le reste est accumulé tel quel.

Pourquoi comme ça

Une simple quote à l'intérieur de guillemets doit rester un caractère ordinaire. Si on testait seulement *est-ce un caractère de quote*, alors **echo "il a dit 'salut'"** ouvrirait et refermerait des états au hasard. C'est pour ça que la fermeture compare au caractère qui a ouvert.

Le piège

append_char(lex, c) ajoute la quote **dans** le token. On la garde volontairement : l'expandre en aura besoin pour savoir si un dollar doit être expansé.

Le tampon dynamique, correction du 23 septembre

```
15 static int grow_buffer(t_lex *lex)
16 {
17     char *bigger;
18
19     bigger = malloc(lex->cap * 2);
20     if (!bigger)
21     {
22         free(lex->buffer);
23         lex->buffer = NULL;
24         return (0);
25     }
26     ft_memcpy(bigger, lex->buffer, lex->len + 1);
27     free(lex->buffer);
28     lex->buffer = bigger;
29     lex->cap = lex->cap * 2;
30     return (1);
31 }
32
33 void append_char(t_lex *lex, char c)
```

```

34 {
35 if (!lex->buffer)
36 return ;
37 if (lex->len + 2 > lex->cap && !grow_buffer(lex))
38 return ;
39 lex->buffer[lex->len] = c;
40 lex->len++;
41 lex->buffer[lex->len] = '\0';
42 }

```

Ce que ça fait

Le tampon du token grandit par doublement, à partir de 64 octets. La longueur courante est gardée dans **lex->len** au lieu d'être recalculée.

Pourquoi comme ça

La version d'origine avait **char buffer[4096]** dans la structure et écrivait sans vérifier la borne : tout mot de 4096 caractères faisait segfaulter le shell. Elle appelait aussi **ft_strlen** à chaque caractère, ce qui rendait le découpage quadratique. Les deux problèmes disparaissent avec un compteur et une capacité.

Le piège

Si le **malloc** échoue, on met **buffer** à NULL plutôt que de tronquer en silence. **lex_tokenize** le détecte et abandonne la ligne. Tronquer serait pire qu'échouer : le shell exécuterait une commande que l'utilisateur n'a pas tapée.

L'expander, par Axel

C'est le module le plus interrogé en soutenance, parce que c'est celui où l'ordre des opérations est une décision et non une évidence.

Le point d'entrée, en trois lignes

```

1 int exp_run(t_shell *shell)
2 {
3     if (expand_tokens(shell))
4         return (1);
5     return (exp_strip_tokens(shell->tokens));
6 }

```

Ce que ça fait

D'abord on remplace les variables dans tous les tokens, ensuite seulement on retire les quotes délimitantes.

Pourquoi comme ça

L'inverse casse tout. Avec **echo '\$USER'**, si on retire les quotes d'abord, le token devient **\$USER** et l'expansion le remplace par la valeur. Le sujet et la section Surprise de la grille exigent le texte littéral. Cet ordre est la raison d'être du module.

L'expansion, consciente des quotes

```

89 char *exp_var_replace(t_shell *shell, char *str, int respect_quotes)
90 {
91     char *res;
92     t_lex_state state;
93     int i;
94     int n;
95
96     res = ft_strdup("");
97     state = STATE_NORMAL;
98     i = 0;
99     while (res && str[i])
100     {
101         if (respect_quotes)

```

```

102 update_state(&state, str[i]);
103 if (str[i] == '$' && state != STATE_SQUOTE)
104     i += expand_one(shell, str + i, &res);
105 else
106     {
107         n = plain_len(str + i, state, respect_quotes);
108         append(&res, str + i, n);
109         i += n;
110     }
111 }
112 return (res);
113 }

```

Ce que ça fait

On reparcourt la chaîne en jouant l'automate des quotes. Un dollar déclenche une expansion sauf si on est entre simples quotes. Sinon, on copie d'un bloc tous les caractères littéraux consécutifs.

Pourquoi comme ça

Le paramètre **respect_quotes** vaut 0 pour un corps de heredoc. Là, les quotes sont du texte et non de la structure, donc une apostrophe ne doit pas couper l'expansion. C'est le même code pour les deux usages, avec un seul drapeau.

Le piège

plain_len a été ajouté le 23 septembre. Avant, **ft_strjoin** était appelé une fois par caractère, et chaque appel recopiait toute la chaîne accumulée : le coût était donc en n au carré. Une ligne de 100 000 caractères mettait 28 secondes. Elle met désormais moins d'une seconde.

Le piège du malloc de taille zéro

```

22 static void append(char **res, char *add, int len)
23 {
24     char *piece;
25     char *joined;
26
27     if (!*res || len == 0)
28         return ;
29     piece = ft_substr(add, 0, len);

```

Ce que ça fait

Si la longueur à coller est nulle, on sort tout de suite.

Pourquoi comme ça

Notre **ft_substr** renvoie NULL pour une longueur de zéro. Sans ce test, on prendrait ce NULL pour un échec de malloc, on mettrait **res** à NULL, et **echo \$INEXISTANT** n'afficherait rien tout en rendant 1. Le bug a coûté une demi-journée : les tests unitaires le montraient, pas les tests d'intégration.

Le piège

Si un correcteur demande un exemple de bug difficile, c'est celui-là qu'il faut raconter. Il montre que vous connaissez votre libft et que vous savez lire une valeur de retour ambiguë.

L'exécuteur et le pipeline, par Davi

Axel, c'est cette page qu'il faut savoir redire sans la lire.

Un étage de pipeline

```

15 static pid_t spawn_stage(t_shell *shell, t_cmd *cmd, int prev_read,
16 int *next_read)
17 {
18     int p[2];

```

```

19  pid_t pid;
20
21  *next_read = -1;
22  if (cmd->next)
23  pipe(p);
24  pid = fork();
25  if (pid == 0)
26  {
27  child_dup_pipes(prev_read, p, cmd->next != NULL);
28  pipe_child_run(shell, cmd);
29  }
30  if (prev_read != -1)
31  close(prev_read);
32  if (cmd->next)
33  {
34  close(p[1]);
35  *next_read = p[0];
36  }
37  return (pid);
38  }

```

Ce que ça fait

Le tube est créé **avant** le fork, sinon le fils ne le verrait pas. Le fils branche ses descripteurs puis part exécuter. Le père referme immédiatement ce qui ne le concerne plus et garde l'extrémité de lecture pour l'étage suivant.

Pourquoi comme ça

Les **close** du père sont la partie qu'on oublie et qui bloque tout. Tant qu'un seul processus garde l'extrémité d'écriture ouverte, le lecteur à l'autre bout ne reçoit jamais sa fin de fichier et reste bloqué dans **read** pour toujours.

Le piège

Question classique : *pourquoi ne pas attendre chaque fils juste après l'avoir créé ?* Parce que le tube a une capacité finie, de l'ordre de 64 kilooctets. Si la première commande écrit plus que ça et que personne ne lit, elle se bloque, et vous attendez un fils qui n'avancera jamais. Il faut forker tout le pipeline puis attendre.

Récupérer le code de sortie

```

15  int exec_wait(pid_t pid)
16  {
17  int status;
18
19  if (waitpid(pid, &status, 0) < 0)
20  return (1);
21  if (WIFEXITED(status))
22  return (WEXITSTATUS(status));
23  if (WIFSIGNALED(status))
24  return (128 + WTERMSIG(status));
25  return (1);
26  }

```

Ce que ça fait

waitpid rend un entier encodé. **WIFEXITED** dit si le fils est sorti normalement et **WEXITSTATUS** en extrait le code. **WIFSIGNALED** dit s'il a été tué, et on rend alors **128 plus le numéro du signal**.

Pourquoi comme ça

Le **status** de **waitpid** n'est pas un code de sortie : c'est un entier où le code, le signal et l'information de core dump sont rangés dans des bits différents. Le lire directement donnerait des valeurs absurdes, comme 256 pour un exit 1.

Le piège

Le 130 de ctrl-C, c'est 128 plus 2. Le 131 de ctrl-backslash, c'est 128 plus 3. Si on vous demande d'où sort le 130, c'est la réponse.

Pourquoi un builtin seul reste dans le père

```
41 void exec_run(t_shell *shell)
42 {
43     t_cmd *cmd;
44
45     cmd = shell->cmds;
46     if (!cmd)
47         return ;
48     if (cmd->next)
49     {
50         exec_pipeline(shell);
51         return ;
52     }
53     if (cmd->argv && cmd->argv[0] && !is_builtin(cmd->argv[0]))
54         exec_single(shell, cmd);
55     else
56         shell->last_exit = run_in_parent(shell, cmd);
57 }
```

Ce que ça fait

Trois chemins. Un pipeline part dans **exec_pipeline**, une commande externe seule dans **exec_single**, un builtin seul reste dans le processus courant.

Pourquoi comme ça

Un **cd** exécuté dans un fils changerait le répertoire du fils, qui meurt aussitôt. Le père n'aurait pas bougé. Même chose pour **export** et **unset** : leur effet doit survivre à la commande. En revanche, dans un pipeline, bash lui-même les met dans un fils, donc **cd /tmp | echo hi** ne change rien au répertoire. On reproduit ce comportement.

Le piège

run_in_parent sauvegarde stdin et stdout avec **dup** avant d'appliquer les redirections et les restaure ensuite. Sans ça, un **echo salut > f** laisserait la sortie du shell branchée sur le fichier pour le reste de la session.

Les heredocs et les signaux, par Davi

Annuler un heredoc avec ctrl-C

```
15 static void sig_handler_heredoc(int signum)
16 {
17     (void)signum;
18     g_signal = SIGINT;
19     write(STDOUT_FILENO, "\n", 1);
20 }
21
22 void setup_heredoc_signals(void)
23 {
24     struct sigaction sa;
25
26     sa.sa_handler = sig_handler_heredoc;
27     sigemptyset(&sa.sa_mask);
28     sa.sa_flags = 0;
29     sigaction(SIGINT, &sa, NULL);
30     sa.sa_handler = SIG_IGN;
31     sigaction(SIGQUIT, &sa, NULL);
32 }
```

Ce que ça fait

Pendant la lecture d'un heredoc, SIGINT note son passage dans la globale et affiche un retour à la ligne.

Pourquoi comme ça

Le détail qui fait tout, c'est **sa_flags = 0**. Le gestionnaire du prompt, lui, utilise **SA_RESTART**. Ici on n'en veut surtout pas : sans **SA_RESTART**, le **read** bloquant qui lit la ligne du heredoc est interrompu et renvoie **-1** avec **errno = EINTR**. C'est ce retour qui permet à **read_heredoc_body** de rendre la main et à **collect_one_heredoc** d'annuler.

Le piège

Avec **SA_RESTART**, le read serait automatiquement relancé après le gestionnaire et le ctrl-C n'aurait aucun effet visible. C'est exactement le genre de détail que le correcteur teste sans le formuler.

Vider le tampon de readline après ctrl-C

```
17 static void sig_handler_prompt(int signum)
18 {
19     (void)signum;
20     g_signal = SIGINT;
21     write(STDOUT_FILENO, "\n", 1);
22     rl_on_new_line();
23     rl_replace_line("", 0);
24     rl_redisplay();
25 }
```

Ce que ça fait

Trois appels de readline, dans cet ordre. On déclare qu'on est sur une nouvelle ligne, on remplace le contenu de la ligne en cours par une chaîne vide, on redessine le prompt.

Pourquoi comme ça

La grille exige que le tampon soit vide après ctrl-C : *press Enter to make sure nothing from the previous line is executed*. Sans **rl_replace_line**, readline garderait ce que l'utilisateur avait tapé et l'enverrait à l'exécution au prochain Entrée. C'est une commande que personne n'a validée qui partirait.

Le piège

rl_replace_line n'existe pas dans la libedit d'Apple. C'est pour ça que le Makefile pointe vers la readline de Homebrew sur macOS. Sans ce bloc, le projet ne compile pas sur Mac, donc pas sur la machine de l'évaluation.

Partie 3

Qui a écrit quoi

Mesuré par **git blame** sur les 48 fichiers du dépôt, le 23 septembre. La colonne *auteur* est celui qui détient la majorité des lignes. Servez-vous en pour la démo croisée : **chacun révise la colonne de l'autre**.

Zone	Auteur principal	Contenu
src/lexer/, 6 fichiers	Axel	Automate de quotes, découpage, opérateurs, liste de tokens
src/parser/ cmd_list, parse_*, syntax_check	Axel	Tokens vers liste de t_cmd, redirections, validation
src/parser/parse_heredoc*.c	Davi	Lecture des heredocs, fichiers temporaires, expansion du corps
src/expander/, 5 fichiers	Axel	\$VAR, \$?, retrait des quotes, ordre des opérations
src/executor/exec_run.c	Axel	Aiguillage : pipeline, commande externe, ou builtin dans le père
src/executor/, 7 autres fichiers	Davi	fork, pipe, dup2, résolution du PATH, attente, codes de sortie
src/redirections/redir_files.c	Davi	open avec les bons drapeaux, dup2, fermeture des descripteurs
src/signals/, 2 fichiers	Davi	Gestionnaires pour le prompt, l'exécution et le heredoc
src/env/, 3 fichiers	Davi	Copie de envp en liste chaînée, accès, retour en tableau
src/builtins/ cd, env, export, unset	Davi	Les quatre builtins qui touchent à l'environnement
src/builtins/ echo, pwd, exit	Axel	Les trois builtins sans effet de bord sur l'environnement
src/utls/ ut_init_data.c, ut_str.c	Axel	run_line, le chaînage complet, et les aides sur les chaînes
src/utls/ ut_cleanup.c, ut_error.c	Davi	Libération globale, messages d'erreur sur le descripteur 2
src/main.c	Davi	Boucle REPL, readline, historique, sortie sur ctrl-D
includes/minishell.h	Écrit à deux	Structures et prototypes. 122 lignes de Davi, 90 d'Axel
tests/, 51 fichiers	Axel	16 suites, 414 assertions, comparaison systématique avec bash

Le plan de révision croisée, une heure à deux

Axel explique à Davi : l'automate à trois états du lexer, pourquoi les quotes sont conservées jusqu'à l'expander, l'ordre expansion puis retrait dans exp_run, et le bug de ft_substr à longueur zéro.

Davi explique à Axel : pourquoi le tube est créé avant le fork, pourquoi le père ferme ses extrémités tout de suite, pourquoi tous les forks précèdent la première attente, le rôle de WIFEXITED contre WIFSIGNALED, et pourquoi le gestionnaire de heredoc n'a pas SA_RESTART.

Ensuite vous **inversez** : chacun redit la partie de l'autre, sans le code sous les yeux. Ce qui ne ressort pas est ce qu'il faut relire.

La campagne adversariale du 23 septembre

Huit agents ont attaqué le shell en parallèle, un par axe de la grille, en comparant systématiquement à bash 5.2 et en pilotant un vrai pseudo-terminal pour les signaux. Environ 900 cas testés. Chaque trouvaille est ensuite passée devant un vérificateur indépendant dont la consigne était de la **réfuter**.

Trouvailles vérifiées	Confirmées	Rejetées	Crash	Bloquantes
54	14	40	0 après correction	0 après correction

Corrigé le jour même

Ce qui n'allait pas	Effet	Correctif
Tampon du lexer figé à 4096 octets sur la pile, écriture sans contrôle de borne	Segfault dès qu'un mot dépasse 4095 caractères. Note éliminatoire.	Tampon dynamique par doublement, longueur suivie, échec de malloc géré
exp_var_replace collait un caractère à la fois avec ft_strjoin	Quadratique. 28 s pour une ligne de 100 000 caractères, plus de 40 s à 200 000.	plain_len regroupe les caractères littéraux. Moins d'une seconde à 1 000 000
resolve_in_path s'arrêtait au premier candidat du bon nom	Un fichier non exécutable ou un répertoire homonyme à gauche masquait le vrai binaire. La grille teste l'ordre du PATH.	On continue jusqu'à trouver un exécutable. 126 si on en a croisé un sans droit, 127 sinon

La leçon à retenir de ces trois bugs

Aucun des trois n'était attrapé par les 399 assertions existantes, et pourtant les suites étaient vertes. Pire, un test unitaire **encodait** le comportement fautif du PATH, donc il défendait le bug.

Si un correcteur vous demande comment vous savez que votre code est juste, la bonne réponse n'est pas *on a des tests*. C'est *on a comparé à bash, cas par cas, et on a ajouté un test quand on a trouvé un écart*. Un test vert prouve que le code fait ce que le test attend, pas qu'il est correct.

Les cinq points moyens que nous n'avons pas corrigés

Aucun n'est éliminatoire, aucun n'est testé par la grille, et chaque modification de l'exécuteur à six jours du rendu coûte plus en risque qu'elle ne rapporte. Ils sont listés ici pour que vous ne soyez pas pris au dépourvu.

Le point	Ce que vous répondez
report_signal_death compare le code aplati à 128 plus SIGQUIT au lieu de tester WIFSIGNALED	Un programme qui sort normalement avec 131 déclenche donc un « Quit (core dumped) » qui n'a pas lieu d'être, et 130 une ligne vide parasite. Le correctif propre est de faire remonter le status brut de waitpid jusqu'à report_signal_death. On l'a identifié, mesuré, et choisi de ne pas y toucher si près du rendu.
Les redirections ne sont pas appliquées quand la commande est introuvable (quatre facettes de la même cause)	resolve_path est appelé avant apply_redirs, donc quand la résolution échoue on sort sans jamais ouvrir les fichiers. Bash applique les redirections dans le fils, puis échoue sur l'execve, ce qui tronque quand même le fichier de sortie. Le correctif serait de forker d'abord et de résoudre dans le fils.

Les écarts avec bash que nous assumons

Le sujet est explicite : *You should limit yourself to the subject description. Anything that is not asked is not required*. Les points ci-dessous sont de vraies différences avec bash, mais hors du périmètre obligatoire. Si le correcteur les soulève, la réponse est la même à chaque fois : **ce n'est pas demandé**, et nous savons pourquoi nous ne l'avons pas fait.

Ce que bash fait et nous pas	Ce que vous répondez
\${VAR} avec accolades	Le sujet demande l'expansion de \$VAR et de \$? . Les accolades sont une syntaxe de substitution de paramètre, avec les valeurs par défaut et les sous-chaînes. Ce n'est pas dans la partie obligatoire.

Ce que bash fait et nous pas	Ce que vous répondez
Découpage en mots après expansion : une variable contenant des espaces donne plusieurs mots	Le sujet ne le demande pas et la plupart des minishell ne le font pas. Cela impliquerait de redécouper après expansion, donc de relancer le lexer sur le résultat. Nous avons choisi de ne pas le faire pour garder une pipeline à sens unique.
\$\$, \$0, \$@, \$#, \$*	Paramètres spéciaux et positionnels. Le sujet n'exige que \$VAR et \$?.
<<- , le heredoc qui mange les tabulations	Le sujet demande <<, pas <<-.
~ et ~utilisateur	L'expansion du tilde n'est pas au programme de la partie obligatoire.
Prompt de continuation sur une quote non fermée	Bash ouvre un prompt secondaire. Le sujet ne demande pas le multiligne, nous rendons une erreur de syntaxe avec \$? à 2.
cd garde le chemin logique à travers un lien symbolique	Nous appelons getcwd, qui rend le chemin physique. C'est la vérité du noyau. Le sujet ne tranche pas.
Un fichier sans shebang est relancé via /bin/sh	Bash rattrape ENOEXEC. Nous rendons une erreur. Cas limite hors partie obligatoire.

La phrase qui clôt le sujet

C'est une vraie différence avec bash, nous l'avons mesurée. Le sujet dit de s'en tenir à ce qui est demandé, et ce point n'y est pas. Nous savons comment nous l'implémenterions si c'était au programme.

Puis vous expliquez comment. C'est cette deuxième phrase qui fait la différence entre une lacune et un choix.

Les questions pièges, et vos réponses

Combien de variables globales, et pourquoi ?

Une seule, **int g_signal**, dans `src/signals/signals.c`. Un gestionnaire de signal ne reçoit qu'un numéro et ne rend rien : il ne peut communiquer avec le reste du programme que par une variable de portée globale. On y met uniquement le numéro du signal reçu. Tout le reste de l'état du shell, l'environnement, le dernier code de sortie, les tokens, vit dans la structure **t_shell** passée en paramètre.

L'appui concret

Le sujet interdit qu'une globale donne accès à autre chose que le numéro du signal. La nôtre est un int, pas un pointeur, donc elle ne peut rien donner d'autre par construction.

Pourquoi expander avant de retirer les quotes, et pas l'inverse ?

Parce que les quotes portent l'information qui décide si l'expansion doit avoir lieu. Une fois retirées, cette information est perdue. Avec **echo '\$USER'**, si on retire d'abord, le token devient `$USER` et l'expansion le remplace par la valeur, alors qu'il faut afficher le texte. C'est exactement ce que teste la section Surprise de la grille.

L'appui concret

`exp_run` tient en trois lignes justement pour que cet ordre soit lisible d'un coup d'œil.

Pourquoi forker tous les étages du pipeline avant la première attente ?

Parce qu'un tube a une capacité finie, de l'ordre de 64 kilooctets. Si on attend le premier fils avant de créer le second, et que le premier écrit plus que la capacité du tube, il se bloque sur son write : personne ne lit à l'autre bout, puisque le lecteur n'existe pas encore. Le shell attend alors un fils qui n'avancera jamais. On crée donc tous les processus, puis on les attend.

L'appui concret

C'est aussi pour ça que le père ferme ses extrémités de tube immédiatement : tant qu'un processus garde l'écriture ouverte, le lecteur ne reçoit jamais sa fin de fichier.

Pourquoi un builtin seul ne part-il pas dans un fils ?

Parce que son effet doit survivre à la commande. Un **cd** dans un fils change le répertoire du fils, qui meurt aussitôt : le shell n'a pas bougé. Même raisonnement pour `export` et `unset`. En revanche, dans un pipeline, `bash` lui-même les met dans un fils, donc **cd /tmp | echo hi** ne change rien. Nous reproduisons ce comportement.

L'appui concret

`run_in_parent` sauvegarde `stdin` et `stdout` avec `dup` avant d'appliquer les redirections, et les restaure après, sinon un `echo` vers un fichier redirigerait la sortie du shell pour toute la session.

D'où sort le 130 de ctrl-C ?

De **128 plus WTERMSIG(status)**. `SIGINT` vaut 2, donc 130. `SIGQUIT` vaut 3, donc 131. C'est la convention POSIX pour distinguer un processus tué par un signal d'un processus sorti normalement avec un petit code. On lit le status de `waitpid` avec `WIFEXITED` et `WIFSIGNALED`, jamais directement : c'est un entier où le code, le signal et le core dump occupent des bits différents.

L'appui concret

Au prompt, quand `ctrl-C` n'a tué aucun fils, c'est `check_prompt_signal` dans `main` qui pose 130 à partir de la globale.

Et les fuites mémoire ?

Zéro **definitely lost** et zéro **indirectly lost**, vérifié sous valgrind sans aucun fichier de suppression, sur 31 scénarios. Les blocs **still reachable** qui restent viennent tous de libreadline et de libtinfo : ce sont les tables de touches et le terminfo, alloués une fois à l'initialisation et gardés dans des variables globales de la bibliothèque. Le sujet, page 9, dit que les fuites de readline ne sont pas de notre responsabilité.

L'appui concret

Le script tests/leaks/leaks_basic.sh rejoue ces scénarios en une commande et échoue sur le moindre bloc definitely lost ou indirectly lost, ce qui nous évite de réintroduire une fuite sans le voir.

Comment gérez-vous l'échec d'un malloc ?

Chaque fonction qui alloue rend NULL en cas d'échec, et l'appelant remonte. Dans l'expand, **exp_var_value** rend **toujours** de la mémoire allouée, même pour une variable inexistante ou vide : ainsi l'appelant libère sans condition, sans avoir à se demander qui possède quoi. Dans le lexer, un échec d'agrandissement du tampon met le pointeur à NULL, ce que lex_tokenize détecte pour abandonner la ligne au lieu de la tronquer en silence.

L'appui concret

Tronquer serait pire qu'échouer : le shell exécuterait une commande que l'utilisateur n'a pas tapée.

Pourquoi une liste chaînée pour l'environnement plutôt qu'un tableau ?

Parce que unset doit retirer un élément au milieu et export doit en ajouter, sans réallouer ni décaler tout le reste. Le tableau n'est refabriqué qu'au moment où on en a vraiment besoin, c'est-à-dire juste avant l'execve, par env_to_array, puis libéré.

L'appui concret

Nous copions envp au démarrage et ne pointons jamais dessus : ce tableau appartient au système.

Qu'est-ce qui vous a pris le plus de temps ?

Le bug de ft_substr avec une longueur de zéro. Notre libft rend NULL dans ce cas, ce que nous prenions pour un échec d'allocation, donc **echo \$INEXISTANT** a n'affichait rien et rendait 1. Les tests unitaires le montraient, les tests d'intégration non, parce qu'ils ne regardaient que la sortie. Depuis, la règle est de comparer à bash plutôt que de comparer à ce que nous attendons.

L'appui concret

C'est la réponse à donner si on vous demande un exemple de bug difficile : elle montre que vous connaissez votre libft et que vous savez lire une valeur de retour ambiguë.

La checklist du jour J

Avant que le correcteur clone

1. Faites un **git clone** dans un dossier vide, puis **make**. C'est ce que le correcteur va faire, faites-le une dernière fois vous-mêmes. Un fichier oublié dans le `.gitignore` et c'est fini.
2. Vérifiez qu'aucun alias n'est en place dans votre shell. La grille demande explicitement au correcteur de chercher des alias malicieux. Si vous en avez un innocent, expliquez-le avant qu'il le trouve.
3. **make -n | grep -- -Wall** pour montrer les drapeaux sans avoir à ouvrir le Makefile.
4. **make** une seconde fois : *Nothing to be done*, donc pas de relink.
5. **norminette src/ includes/ libft/** : zéro erreur.
6. Les 16 suites de tests, 414 assertions, en une commande. Proposez-la, ne l'imposez pas : la grille demande de revoir ensemble tout script de test avant de s'en servir.

Pendant la soutenance

Ne touchez jamais au clavier du correcteur. S'il tape une commande qui vous surprend, laissez-la finir. Un shell qui plante parce que vous avez voulu l'empêcher de taper, c'est le même zéro.

Si ça plante, dites-le avant lui. Nommer soi-même le problème et expliquer pourquoi vaut toujours mieux que le laisser découvrir et vous voir chercher.

Ne dites jamais « je ne sais pas, c'est l'autre qui l'a fait ». C'est la phrase qui coûte le plus cher en soutenance de binôme. Si vous ne savez pas, dites ce que vous savez du module et raisonnez à voix haute : le correcteur note la compréhension, pas la récitation.

Le sujet est votre argument. Pour tout ce qui n'est pas demandé, citez-le. C'est écrit noir sur blanc : *Anything that is not asked is not required*.

La timeline du projet

Du 26 mai au 23 septembre 2026, 96 commits, deux personnes. Utile si le correcteur demande comment vous vous êtes organisés.

Période	Ce qui a été fait	Qui
26 mai	Organisation du dépôt, guide d'équipe, squelette de tests, README	Axel
4 juin	Fondations : structures, squelette de main, Makefile	Davi
15 juin au 28 juillet	Environnement : copie de envp, API get, set et find, retour en tableau, affichage	Davi
2 au 15 juillet	Lexer : découpage sur les blancs, détection des opérateurs	Axel
1er au 11 août	Lexer : automate de quotes à trois états	Axel
12 au 27 août	Messages d'erreur, libération mémoire, tests d'environnement	Davi
8 au 17 septembre	Parser : tokens vers t_cmd, redirections. Builtins echo, pwd, exit. Chaînage REPL et validation syntaxique	Axel
11 au 18 septembre	Exécuteur : résolution du PATH, fork, redirections, pipeline, export, heredoc	Davi
21 septembre	Expander complet en quatre PR : longueur du nom, valeur, remplacement, tests de bout en bout	Axel
21 septembre	unset, tests de pipes et de redirections, correction de cd pour macOS, signaux	Davi
22 septembre	Fuites de descripteurs, bug de heredoc, tabulation comme séparateur	Davi
23 septembre	Correction du build macOS, nettoyage des fichiers vides, README conforme au chapitre V, campagne adversariale et ses trois correctifs	Axel

S'il ne fallait retenir que six phrases

1. Une seule globale, un int, parce qu'un gestionnaire de signal ne peut écrire nulle part ailleurs.
2. Les quotes survivent jusqu'à l'expander, parce qu'elles décident si un dollar s'expande.
3. On expande puis on retire les quotes, jamais l'inverse.
4. On forke tout le pipeline avant d'attendre, parce qu'un tube a une capacité finie.
5. Un builtin seul reste dans le père, parce que cd doit survivre à la commande.
6. 128 plus le numéro du signal, lu avec WIFSIGNALED, jamais le status brut.